实验四 随机森林分类Fashion-Mnist

姓名　顾凯琪

学号　226001624

学院 信息学院

专业班级 计算机2班

一、 实验目的

1. 学习掌握决策树的基本原理以及使用决策树进行分类的方法；
2. 学习集成学习算法的相关概念，了解装袋法和提升法的原理；
3. 学习掌握将决策树作为弱分类器的集成学习方法——随机森林，了解其基本原理，并基于sklearn框架实现随机森林，并在MNIST数据集上进行训练与分类，验证模型效果；
4. 通过调整随机森林的相关参数，探索模型的一些超参数对模型性能的影响，同时对比随机森林算法与逻辑回归模型的分类性能。

二、 实验环境

操作系统：

Windows 11

集成开发环境：

1. Anaconda3
2. PyCharm 2022.2.2
3. VScode

相关Python库：

1. python3.9
2. numpy
3. matplotlib
4. sklearn

三、 实验内容

1. 基于 Sklearn 框架实现使用随机森林算法对 Fashion-Mnist 数据集的分类任务；
2. 基于 Sklearn 框架实现使用逻辑回归算法对 Fashion-Mnist 数据集的分类任务；
3. 比较这两种方法的准确率。

四、 实验步骤：

1. 数据集导入

本次实验使用随机森林模型进行多分类问题的求解，模型的训练与验证基于 Fashion-MNIST 数据集进行，该数据集容易在互联网上获取，本次实验采用实验指导给出的二进制形式的数据集。

数据集中包含 60000 个训练样本与 10000 个训练样本,每个样本由一张 28*28 像素的图像与一个标签组成，编写代码实现数据集的导入，并进行适当的可视化操作（使用 matplotlib.pyplot 实现）。

2. 使用 sklearn 库实现模型训练

（1）模型定义

Sklearn 库中包含许多集成好的机器学习模型，其中集成学习模型中包含 RandomForestClassifier 类，即为随机森林，已实现封装，通过调用 fit 函数实现模型的训练，predict 函数实现模型的预测。

此外模型包含一系列重要参数，分别代表随机种子、最大子树个数、最大树深度等信息，本次实验主要验证最大字数个数对随机森林的性能影响。

（2）模型训练

对于 sklearn 库，已经完成了基本的模型训练过程封装，只需要进行模型的实例化，调用 fit()函数传入训练数据即可完成训练，调用 predict 函数并传入测试数据的特征即可得到对应的结果。

（3）训练结果评价

由于本次实验进行的为多分类的模型训练，评价指标使用准确率，即预测正

确的样本个数与总样本数之间的比值，模型在测试集上的准确率反应了模型的泛化能力与性能，在此用于描述模型的性能。

3. 使用逻辑回归模型进行多分类

完成随机森林分类后，采用逻辑回归模型对同一数据集进行训练，验证模型性能并进行对比分析。

4. 超参数调整

修改随机森林的最大子树个数并重新训练模型，验证决策树的最大子树个数对随机森林分类性能的影响。

五、 代码提交

1. sklearn 实现随机森林分类

```
"""
-------相关库导入-----
"""
```

```
# 相关库导入
import os
import gzip
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# 加载数据集（从二进制文件进行加载）
def load_mnist(path, kind='train'):
    label_path =os.path.join(path,'{}-labels-idx1-ubyte'.format(kind))
    image_path =os.path.join(path,'{}-images-idx3-ubyte'.format(kind))
    with open(label_path, 'rb') as lp:
        labels = np.frombuffer(lp.read(), dtype=np.uint8, offset=8)
    with open(image_path, 'rb') as ip:
        images = np.frombuffer(ip.read(), dtype=np.uint8, offset=16).re
shape(len(labels), 784)
    return images, labels
# 数据可视化操作
def image_disp(data_list):
```

```
    for i in range(4):
        feature = data_list[i]
        image = feature.reshape(28, 28)
        plt.subplot(2, 2, i+1)
        plt.imshow(image)
        plt.axis('off')
        # plt.tight_layout()
        plt.show()
```

```
"""
-------获取训练集与测试集----
"""
```

```
# 获取训练集与测试集
x_train, y_train = load_mnist('./FashionMNIST/', kind='train')
x_test, y_test = load_mnist('./FashionMNIST/', kind='t10k')
# 显示训练集数据
image_disp(x_train)
```

```
"""
-------模型训练---
"""
```

```
# 随机种子
seed_value = 2025
model = RandomForestClassifier(random_state=seed_value)
# 模型训练
model.fit(x_train, y_train)
# 在训练集、测试集上进行预测
y_train_pred = model.predict(x_train)
y_test_pred = model.predict(x_test)
```

```
"""
-------准确率----
"""
```

```
# 计算模型在测试集、训练集上的准确率
acc_train = (y_train_pred == y_train).sum() / len(y_train)
acc_test = (y_test_pred == y_test).sum() / len(y_test)
acc_train, acc_test
```

2. 逻辑回归分类

```
"""
-------导入和训练预测----
"""
```

```
# 导入 skleran 中的逻辑回归模型
from sklearn.linear_model import LogisticRegression
logistic = LogisticRegression()
# 数据拟合
logistic.fit(x_train, y_train)
# 模型预测
y_train_pred_logistic = logistic.predict(x_train)
y_test_pred_logistic = logistic.predict(x_test)
# 准确率计算
acc_train_logistic=(y_train_pred_logistic==y_train).sum()/len(y_train)
acc_test_logistic=(y_test_pred_logistic==y_test).sum()/len(y_test)
acc_train_logistic, acc_test_logistic
```

3. 模型超参数（最大子树数）调整

```
from sklearn.model_selection import cross_val_score
# 绘制曲线来调整 n_estimators 子树的数量
score_list = []
# 每隔 10 步建立一个随机森林，分别进行训练，使用集成的交叉验证函数来进行评估
for i in range(0,200,10):
    model = RandomForestClassifier(n_estimators=i+1, random_state=90)
    score = cross_val_score(model, x_train, y_train, cv=10).mean()
    score_list.append(score)
    print(score)
    score_max = max(score_list)
    print('最优得分：{}'.format(score_max),
          '包含子树数量为：{}'.format(score_list.index(score_max)*10+1))
    # 绘制学习曲线
    x = np.arange(1,201,10)
    plt.plot(x, score_list, 'r-')
    plt.show()
```

六、 实验结果与分析

该部分将对实验代码产出的相关结果与图像进行记录与分析。需要注意的是：下部分实验图片，有部分来自 notebook，有部分来自 pycharm 源码，原因是 ipynb 的格式便于单步展示结果，完整的训练过程我使用 pycharm 进

行展示。

1. 数据可视化

由于给定的数据为 28*28 的图像，输入为 784 维的特征，需要进行 reshape 操作，最后使用 matplotlib 库进行绘图，选取前四个样本数据，绘图结果如图 6.1 所示。可以看到数据内容为各种服饰，需要训练模型实现对服饰的分类。

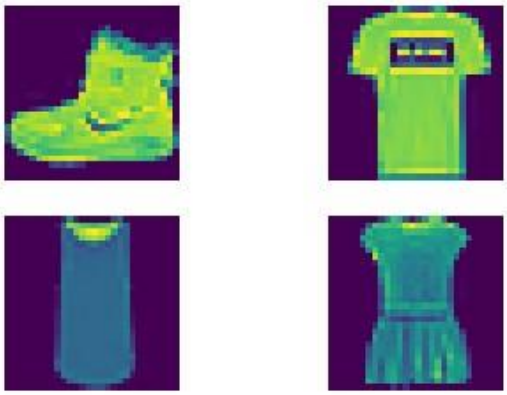


图 6.1 Fashion-MNIST 数据可视化

2. 随机森林分类

使用 sklearn 库的 RandomForestClassifier 类进行模型创建，设置随机种子为 2025，其余参数均为默认值，调用 fit 函数进行模型的训练，随后使用 predict 函数使模型预测测试集的数据，最后计算模型的准确率，如图 6.2 所示。可以看到模型在训练集上的准确率经过多轮迭代，准确率大概停留在 88.4%，同时测试集的准确率表现出较好的泛化性。

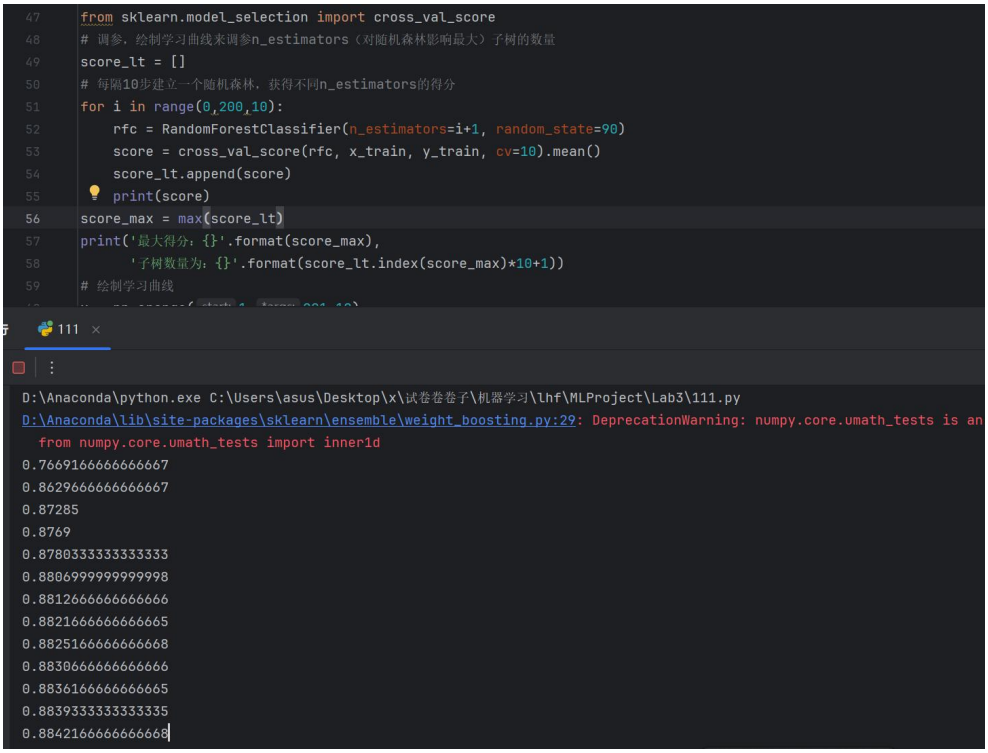


图 6.2 模型训练与准确率计算（默认超参数）

3. 训练时长分析

在实际训练过程中，模型的训练时间超过了 20min，后发现默认的随机森林训练参数只是用 1 核心进行训练，因此产生了较长的训练耗时，并且因为精度要求，我采用了 10 折交叉验证，最大步长为 10，相当于我需要重建 2000 次决策树，这将消耗极大的时间，因此，可以对模型适当优化，调用 8 核心，使用 5 折验证。

```
    # 交叉验证（并行处理，需要放在主函数中）
    score = cross_val_score(rfc, x_train, y_train, cv=5, n_jobs=8).mean()
    print("10折交叉验证得分：", score)

if __name__ == "__main__":
    main()
```

4. 逻辑回归分类

使用 sklearn 库中实现的逻辑回归模型，并使用原数据进行训练与测试，计算模型在测试集上的准确率，结果如图 6.3 所示。

从结果可以看出，相较随机森林准确率较低，为 84.12%。这是由模型的复杂度决定的，数据的特征维数为 786，随机森林会对其进行一定程度的筛选与树的剪枝，以达到较好的判别效果，而逻辑回归则是尝试最小化损失函数，容易造成

过拟合，而无法达到较好的泛化性能。

```
In 65
from sklearn.linear_model import LogisticRegression
logistic = LogisticRegression()
logistic.fit(x_train, y_train)
y_train_pred_logistic = logistic.predict(x_train)
y_test_pred_logistic = logistic.predict(x_test)
acc_train_logistic = (y_train_pred_logistic == y_train).sum() / len(y_train)
acc_test_logistic = (y_test_pred_logistic == y_test).sum() / len(y_test)
acc_train_logistic, acc_test_logistic

Out 65   (0.8632, 0.8412)
```

图 6.3 逻辑回归分类的准确率

5. 超参数调整

对于随机森林模型，sklearn 库中提供了一系列模型的超参数，供调用者进行修改，其中 n_estimators（决策树子树最大个数）对模型的性能影响较大，因此逐步调整该参数，尝试寻找最优的子树个数，具体操作为从 1 到 200 遍历子树个数，对于每一个子树个数，重新训练模型并计算预测准确率，记录结果，最终可以绘制以下图像，如图 6.4 所示。

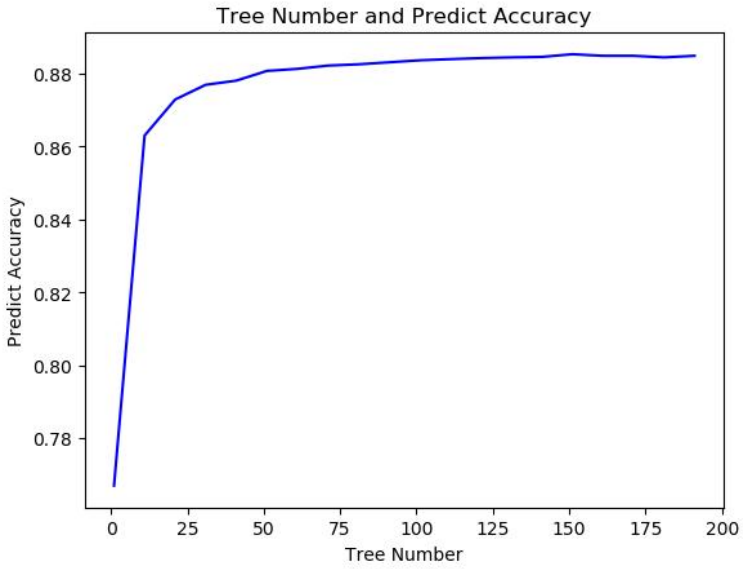


图 6.4 决策树数目与模型准确率的关系

可以发现，模型预测的准确率大体随着随机森林中的决策树的数目的增大而增大，但当随机数数目大于 100 后，模型的准确率提升幅度较小，本质上已经产生了震荡现象，由此可见，再增大决策树的数目已对模型性能无明显贡献，还会

带来明显的模型计算时耗，因此，可以将决策树的数目选定在 75-100 之间，此时模型再测试集上的准确率达到 88.5%上下，已具备可观的泛化性。

七、 理论总结：

本次实验的主要任务是了解并应用两种常见的机器学习模型——决策树和随机森林。通过实验操作和模型对比，我对这两种算法有了更加直观和深入的认识。

决策树是一种以树形结构划分数据的分类模型。它通过不断地选择最优特征进行数据划分，最终形成一棵能够做出预测的“树”。每一个非叶节点代表一个特征的判断条件，而每个叶节点则对应一个分类结果。决策树的优点是模型结构直观、易于理解，同时对数据预处理的要求不高。但它也存在一定的不足，比如容易发生过拟合，对训练数据中的噪声比较敏感。

为了解决决策树容易过拟合的问题，实验中我们还学习了随机森林模型。它是集成学习中的一种方法，实际上就是将多个决策树组合在一起，每棵树都在不同的样本和特征子集上进行训练。最终的分类结果由多数投票决定，这种“众人决定”的方式可以有效提高模型的稳定性和准确率，同时降低过拟合的风险。虽然随机森林牺牲了一些可解释性，但在实际应用中往往表现更优。

除了算法本身，我还通过本次实验了解了训练集和测试集的划分、模型评估指标（如准确率、召回率等），以及如何利用编程工具（如 Python 中的 sklearn 库）实现这些算法。这让我对机器学习的完整流程有了更清晰的认识。

总的来说，这次实验不仅加深了我对决策树和随机森林模型的理解，也提高了我使用编程工具解决实际问题的能力，为后续学习更复杂的模型打下了基础。

八、 讨论、心得

本次实验基于 Python 与 sklearn 库进行了随机森林模型的训练与测试，在 Fashion-MNIST 数据集上进行分类操作，最终设定随机森林模型中的决策树数目为 75 时，模型表现出较好的泛化性能，在测试数据集上达到了 88.5%的准确率。此外实验还对比了随机森林和逻辑回归模型在同一数据集上进行分类操作的性能差异，结果显示随机森林具备更好的分类能力。

通过本次实验，学习了有关决策树模型的基本原理，同时基于决策树的相关知识，进一步学习了集成学习方法中的随机森林，并通过代码编写实现了随机森林的训练，同时也相关实验学习了随机森林模型中的一些超参数对模型参数的影响，通过优化超参数使模型产生更好的性能表现。